PROJETO ACADÊMICO / ENGENHARIA DE SOFTWARE

RPG Literário (BookQuest)

Arquitetura de Software Orientada a Objetos & Especificação de Backend REST

> **Resumo da Proposta:** Transformar o hábito diário de leitura em uma aventura de RPG interativa, inspirada em redes de leitura como o Skoob. Cada livro adicionado à estante atua como um *Chefe (Boss)* cujo HP total equivale ao número de páginas. Cada página lida pelo usuário no mundo real desfere dano no monstro, gerando XP, níveis e recompensas no backend.

1. Fundamentos de POO Aplicados

Pilar de POO	Aplicação Prática no Domínio do Jogo
Abstração	Criação da classe abstrata `EntidadeCombate`, encapsulando mecânicas essenciais de combate (vida máxima, vida atual, recebimento de dano e status de sobrevivência) desacopladas de regras específicas de jogadores ou monstros.
Encapsulamento	Atributos críticos como `vida`, `experiencia` e `paginasLidas` são estritamente privados/protegidos. Apenas métodos de domínio (ex: `receberLeitura()`, `ganharExperiencia()`) podem modificar esses estados com validações de regra de negócio.
Herança	`Leitor` (Jogador) e `InimigoLivro` herdam de `EntidadeCombate`, garantindo reuso limpo de código e consistência conceitual.
Polimorfismo	Classes especializadas como `LivroChefe` podem estender `InimigoLivro` para reagir com mecânicas próprias (ex: contra-ataque passivo caso o leitor fique dias sem ler, ou multiplicadores por gênero literário).

2. Arquitetura do Sistema e Separação de Camadas

A aplicação adota separação estrita de responsabilidades para permitir que o Front-end seja agnóstico (Mobile, Web ou Desktop):

- **Domain (Domínio):** Contém as entidades puras e interfaces de repositório, sem dependência de banco de dados ou Express.
- **Services (Casos de Uso):** Orquestra as regras do jogo (como a realização de um turno de leitura) e integrações de terceiros.
- **Infrastructure (Infraestrutura):** Gerencia a persistência (arquivos JSON ou banco SQLite/Postgres) e chamadas HTTP externas.
- **Presentation (Camada REST):** Expõe rotas HTTP e serialização JSON para consumo pelo Front-end.

3. Modelagem das Entidades (Código do Domínio)

A. Entidade Base e InimigoLivro

```
export abstract class EntidadeCombate {

  constructor(

    public readonly nome: string,

    protected vidaMaxima: number,

    protected vida: number

  ) {}



  public getVida(): number { return this.vida; }

  public getVidaMaxima(): number { return this.vidaMaxima; }

  public estaVivo(): boolean { return this.vida > 0; }



  public receberDano(dano: number): void {

    this.vida = Math.max(0, this.vida - dano);

  }

}



export class InimigoLivro extends EntidadeCombate {

  private paginasLidas: number = 0;

  private status: "QUERO_LER" | "LENDO" | "LIDO" = "QUERO_LER";



  constructor(

    public readonly id: string,

    public readonly titulo: string,

    public readonly autor: string,

    public readonly totalPaginas: number,

    public readonly googleBooksId?: string,
```

```
    public readonly capaUrl?: string
  ) {
    // A vida total do Boss é o número total de páginas do livro
    super(titulo, totalPaginas, totalPaginas);
  }

  public receberLeitura(paginas: number) {
    if (!this.estaVivo()) throw new Error("O livro já foi concluído!");

    const danoReal = Math.min(paginas, this.vida);
    this.paginasLidas += danoReal;
    this.receberDano(danoReal);

    if (this.status === "QUERO_LER") this.status = "LENDO";
    if (!this.estaVivo()) this.status = "LIDO";

    return { danoCausado: danoReal, finalizado: !this.estaVivo() };
  }

  public getPaginasLidas(): number { return this.paginasLidas; }
  public getStatus() { return this.status; }
}
```

B. Jogador (Leitor)

```
export class Leitor extends EntidadeCombate {
  private experiencia: number = 0;

  constructor(
    public readonly id: string,
```

```
    nome: string,
    vidaMaxima: number = 100,
    private danoPorPagina: number = 1
  ) {
    super(nome, vidaMaxima, vidaMaxima);
  }

  public getNivel(): number {
    return Math.floor(this.experiencia / 100) + 1;
  }

  public ganharExperiencia(xp: number): void {
    this.experiencia += xp;
  }

  public getExperiencia(): number { return this.experiencia; }

  public calcularDano(paginas: number): number {
    return paginas * this.danoPorPagina;
  }
}
```

4. Regra de Negócio: Turno de Leitura (BatalhaService)

```
export class BatalhaService {
  public atacarLivro(leitor: Leitor, livro: InimigoLivro, paginasLidas: number) {
    const dano = leitor.calcularDano(paginasLidas);
    const { danoCausado, finalizado } = livro.receberLeitura(dano);

    // 1 página lida = 2 de XP. Bônus de 100 XP ao finalizar um livro!
```

```
    let xpGanho = danoCausado * 2;

    if (finalizado) xpGanho += 100;



    leitor.ganharExperiencia(xpGanho);



    return {
      paginasLidas: danoCausado,
      danoCausado,
      vidaRestanteLivro: livro.getVida(),
      livroDerrotado: finalizado,
      xpGanho,
      nivelAtual: leitor.getNivel()
    };
  }
}
```

5. Integração com API Externa (Google Books)

```
export class GoogleBooksService {

  public async buscarPorTermo(termo: string) {

    const url = `https://www.googleapis.com/books/v1/volumes?q=${encodeURIComponent(termo)}
&maxResults=5`;

    const resposta = await fetch(url);

    const data = await resposta.json();



    if (!data.items) return [];



    return data.items.map((item: any) => ({

      id: item.id,

      titulo: item.volumeInfo.title || "Sem título",
```

```
      autores: item.volumeInfo.authors || ["Autor desconhecido"],
      totalPaginas: item.volumeInfo.pageCount || 100,
      descricao: item.volumeInfo.description,
      capaUrl: item.volumeInfo.imageLinks?.thumbnail
    }));
  }
}
```

6. Especificação das Rotas da API REST

Método & Rota	Finalidade	Payload / Parâmetros
`GET /api/livros/busca?q={nome}`	Pesquisa livros no catálogo do Google Books para extração de metadados e páginas.	Query param: `q` (termo de busca)
`POST /api/estante`	Cadastra um novo livro na biblioteca do usuário (invocando-o como Boss).	Body JSON: `{ id, titulo, autor, totalPaginas, capaUrl }`
`GET /api/estante`	Retorna a lista de livros do leitor (HP atual, HP máximo, status de leitura e nível do jogador).	Nenhum
`POST /api/estante/:id/ler`	Registra sessão de leitura, aplicando dano ao livro e concedendo XP ao jogador.	Body JSON: `{ "paginas": 30 }`